

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)

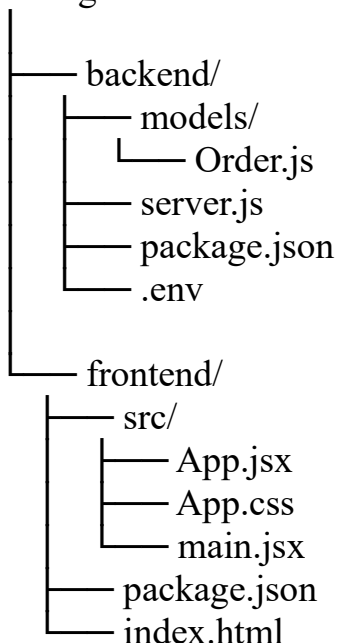
MERN STACK
Practical – MINI Project

Roll No.: T047	Name: Ashwin Tripathi
Class: TYIT	Batch:1
Date of Assignment: 26-09-26	Date/Time of Submission: 26-09-26

Aim: Mini Project for college canteen preordering system

Step 1 : Project Structure...

College-Canteen-PreOrder/



project folder structure → MongoDB setup → Express/Node backend → React frontend → API testing → final output

Step 2 : Tools used.

• React • Axios • Node.js • Express.js • MongoDB.

Step 3: Backend

- a. Npm init -y
- b. Npm install mongoes express cors dotenv
- c. Npm install --save--dev nodemon

Step 4: backend/server.js

Code :

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
require("dotenv").config();
```

```
const Order = require("../models/Order");

const app = express();

app.use(cors());
app.use(express.json());

// MongoDB Connection
mongoose
  .connect(process.env.MONGO_URI)
  .then(() => {
    console.log("MongoDB Connected Successfully");
  })
  .catch((error) => {
    console.log("MongoDB Connection Error:", error.message);
  });

// Food items
const foodItems = [
  {
    id: 1,
    name: "Veg Sandwich",
    price: 50
  },
  {
    id: 2,
    name: "Masala Dosa",
    price: 60
  },
  {
    id: 3,
    name: "Veg Burger",
    price: 70
  },
  {
    id: 4,
    name: "Pizza",
    price: 100
  },
  {
    id: 5,
    name: "Cold Drink",
    price: 30
  }
];
```

```
}  
];
```

```
// GET /food  
app.get("/food", (req, res) => {  
  res.json(foodItems);  
});
```

```
// POST /orders  
app.post("/orders", async (req, res) => {  
  try {  
    const {  
      studentName,  
      rollNumber,  
      course,  
      foodItem,  
      quantity,  
      price  
    } = req.body;  
  
    if (  
      !studentName ||  
      !rollNumber ||  
      !course ||  
      !foodItem ||  
      !quantity ||  
      !price  
    ) {  
      return res.status(400).json({  
        message: "All fields are required"  
      });  
    }  
  }  
}
```

```
const totalPrice = Number(price) * Number(quantity);
```

```
const newOrder = new Order({  
  studentName,  
  rollNumber,  
  course,  
  foodItem,  
  quantity,  
  price,  
  totalPrice  
});
```

```

    });

    const savedOrder = await newOrder.save();

    res.status(201).json({
      message: "Order placed successfully",
      order: savedOrder
    });
  } catch (error) {
    res.status(500).json({
      message: "Error placing order",
      error: error.message
    });
  }
});

// GET /orders
app.get("/orders", async (req, res) => {
  try {
    const orders = await Order.find().sort({ createdAt: -1 });

    res.json(orders);
  } catch (error) {
    res.status(500).json({
      message: "Error retrieving orders",
      error: error.message
    });
  }
});

// Start server
const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});

```

Step 5: Create Order Model.. backend/model/Order.js

Code :

```

const mongoose = require("mongoose");

```

```

const orderSchema = new mongoose.Schema(
  {
    studentName: {

```

```

    type: String,
    required: true
  },

  rollNumber: {
    type: String,
    required: true
  },

  course: {
    type: String,
    required: true
  },

  foodItem: {
    type: String,
    required: true
  },

  quantity: {
    type: Number,
    required: true,
    min: 1
  },

  price: {
    type: Number,
    required: true
  },

  totalPrice: {
    type: Number,
    required: true
  }
},
{
  timestamps: true
}
);

```

module.exports = mongoose.model("Order", orderSchema);

Step 6: Create .env in backend folder

Code :

MONGO_URI=mongodb://localhost:27017/college_canteen
PORT=5000

Step 7: Update backend/package.json

Code : add this code..

```
"scripts": {  
  "start": "node server.js",  
  "dev": "nodemon server.js"  
}
```

Step 8 : run and check api

- Npm run dev
- You will see MongoDB connected successfully
- Localhost:5000
- Search <https://localhost:5000/food>

Step 9 : Start Backend .

```
├── frontend/  
│   ├── src/  
│   │   ├── App.jsx  
│   │   ├── App.css  
│   │   └── main.jsx  
│   ├── package.json  
│   └── index.html
```

- npm create vite@latest frontend -- --template react
- npm install
- npm install axios

Step 10 :Add code App.jsx

Code :

```
import { useEffect, useState } from "react";  
import axios from "axios";  
import "./App.css";
```

```
function App() {  
  const [foodItems, setFoodItems] = useState([]);  
  const [orders, setOrders] = useState([]);  
  
  const [studentName, setStudentName] = useState("");  
  const [rollNumber, setRollNumber] = useState("");  
  const [course, setCourse] = useState("");  
  const [foodItem, setFoodItem] = useState("");  
  const [quantity, setQuantity] = useState(1);  
  const [price, setPrice] = useState(0);  
  
  const API_URL = "http://localhost:5000";
```

```
// Get food items
const getFoodItems = async () => {
  try {
    const response = await axios.get(`${API_URL}/food`);
    setFoodItems(response.data);
  } catch (error) {
    console.log(error);
  }
};
```

```
// Get orders
const getOrders = async () => {
  try {
    const response = await axios.get(`${API_URL}/orders`);
    setOrders(response.data);
  } catch (error) {
    console.log(error);
  }
};
```

```
// Load data when page opens
useEffect(() => {
  getFoodItems();
  getOrders();
}, []);
```

```
// Food selection
const handleFoodChange = (e) => {
  const selectedFood = foodItems.find(
    (item) => item.name === e.target.value
  );

  setFoodItem(e.target.value);

  if (selectedFood) {
    setPrice(selectedFood.price);
  } else {
    setPrice(0);
  }
};
```

```
// Place order
```

```
const placeOrder = async (e) => {
  e.preventDefault();

  if (
    !studentName ||
    !rollNumber ||
    !course ||
    !foodItem ||
    !quantity
  ) {
    alert("Please fill all fields");
    return;
  }

  try {
    await axios.post(`${API_URL}/orders`, {
      studentName,
      rollNumber,
      course,
      foodItem,
      quantity: Number(quantity),
      price: Number(price)
    });

    alert("Order placed successfully!");

    // Clear form
    setStudentName("");
    setRollNumber("");
    setCourse("");
    setFoodItem("");
    setQuantity(1);
    setPrice(0);

    // Refresh orders
    getOrders();
  } catch (error) {
    console.log(error);
    alert("Error placing order");
  }
};

return (
```



```
<div className="container">

  <h1>College Canteen Pre-Order System</h1>

  { /* Food Items */ }
  <section>
    <h2>Available Food Items</h2>

    <div className="food-container">
      { foodItems.map((food) => (
        <div className="food-card" key={food.id}>
          <h3>{food.name}</h3>
          <p>Price: ₹ {food.price}</p>
        </div>
      )) }
    </div>
  </section>

  { /* Order Form */ }
  <section className="form-section">

    <h2>Student Order Form</h2>

    <form onSubmit={placeOrder}>

      <input
        type="text"
        placeholder="Student Name"
        value={studentName}
        onChange={(e) => setStudentName(e.target.value)}
      />

      <input
        type="text"
        placeholder="Roll Number"
        value={rollNumber}
        onChange={(e) => setRollNumber(e.target.value)}
      />

      <input
        type="text"
        placeholder="Course"
        value={course}
```

```

        onChange={ (e) => setCourse(e.target.value)}
      />

      <select
        value={foodItem}
        onChange={handleFoodChange}
      >
        <option value="">Select Food Item</option>

        {foodItems.map((food) => (
          <option key={food.id} value={food.name}>
            {food.name} - ₹ {food.price}
          </option>
        ))}
      </select>

      <input
        type="number"
        min="1"
        placeholder="Quantity"
        value={quantity}
        onChange={ (e) => setQuantity(e.target.value)}
      />

      <input
        type="text"
        value={
          price
          ? `₹${price * Number(quantity)}`
          : "Total Price"
        }
        readOnly
      />

      <button type="submit">
        Place Order
      </button>

    </form>
  </section>

  { /* Orders */ }
</section>

```

```
<h2>Placed Orders</h2>
```

```
<table>
```

```
  <thead>
```

```
    <tr>
```

```
      <th>Student Name</th>
```

```
      <th>Roll No</th>
```

```
      <th>Course</th>
```

```
      <th>Food Item</th>
```

```
      <th>Quantity</th>
```

```
      <th>Price</th>
```

```
      <th>Total</th>
```

```
    </tr>
```

```
  </thead>
```

```
  <tbody>
```

```
    {orders.map((order) => (
```

```
      <tr key={order._id}>
```

```
        <td>{order.studentName}</td>
```

```
        <td>{order.rollNumber}</td>
```

```
        <td>{order.course}</td>
```

```
        <td>{order.foodItem}</td>
```

```
        <td>{order.quantity}</td>
```

```
        <td>₹ {order.price}</td>
```

```
        <td>₹ {order.totalPrice}</td>
```

```
      </tr>
```

```
    )))
```

```
  </tbody>
```

```
</table>
```

```
</section>
```

```
</div>
```

```
);
```

```
}
```

```
export default App;
```

App.css

```
body {  
  margin: 0;  
  font-family: Arial, sans-serif;  
  background-color: #f4f6f8;  
}
```

```
.container {  
  width: 90%;  
  margin: auto;  
  padding: 20px;  
}
```

```
h1 {  
  text-align: center;  
  background-color: #222;  
  color: white;  
  padding: 20px;  
  border-radius: 8px;  
}
```

```
h2 {  
  margin-top: 30px;  
}
```

```
/* Food */
```

```
.food-container {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 20px;  
}
```

```
.food-card {  
  background-color: white;  
  width: 180px;  
  padding: 20px;  
  border-radius: 10px;  
  box-shadow: 0 2px 8px #ccc;  
  text-align: center;  
}
```

```
.food-card h3 {
```

```
    margin-bottom: 10px;
}

/* Form */

.form-section {
    background-color: white;
    margin-top: 30px;
    padding: 25px;
    border-radius: 10px;
}

form {
    display: flex;
    flex-direction: column;
    gap: 12px;
    max-width: 500px;
}

input,
select {
    padding: 12px;
    border: 1px solid #ccc;
    border-radius: 5px;
    font-size: 15px;
}

button {
    padding: 12px;
    background-color: #222;
    color: white;
    border: none;
    border-radius: 5px;
    cursor: pointer;
    font-size: 16px;
}

button:hover {
    background-color: #444;
}

/* Table */
```

```

table {
  width: 100%;
  margin-top: 20px;
  border-collapse: collapse;
  background-color: white;
}

th,
td {
  border: 1px solid #ddd;
  padding: 12px;
  text-align: center;
}

th {
  background-color: #222;
  color: white;
}

tr:nth-child(even) {
  background-color: #f2f2f2;
}

```

Main.jsx

```

import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./App.css";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);

```

Step 10: run both backend and frontend

Syntax

Backend server:

```
cd College-Canteen-PreOrder/backend
```

```
npm run dev
```

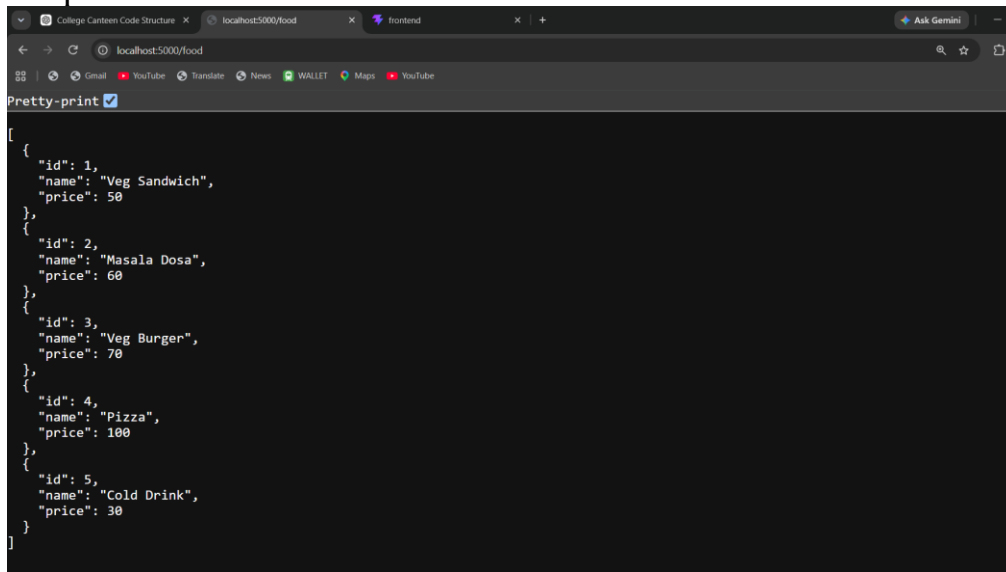
frontend server :

```
cd College-Canteen-PreOrder/frontend
```

```
npm run dev
```

Step 11: Api testing

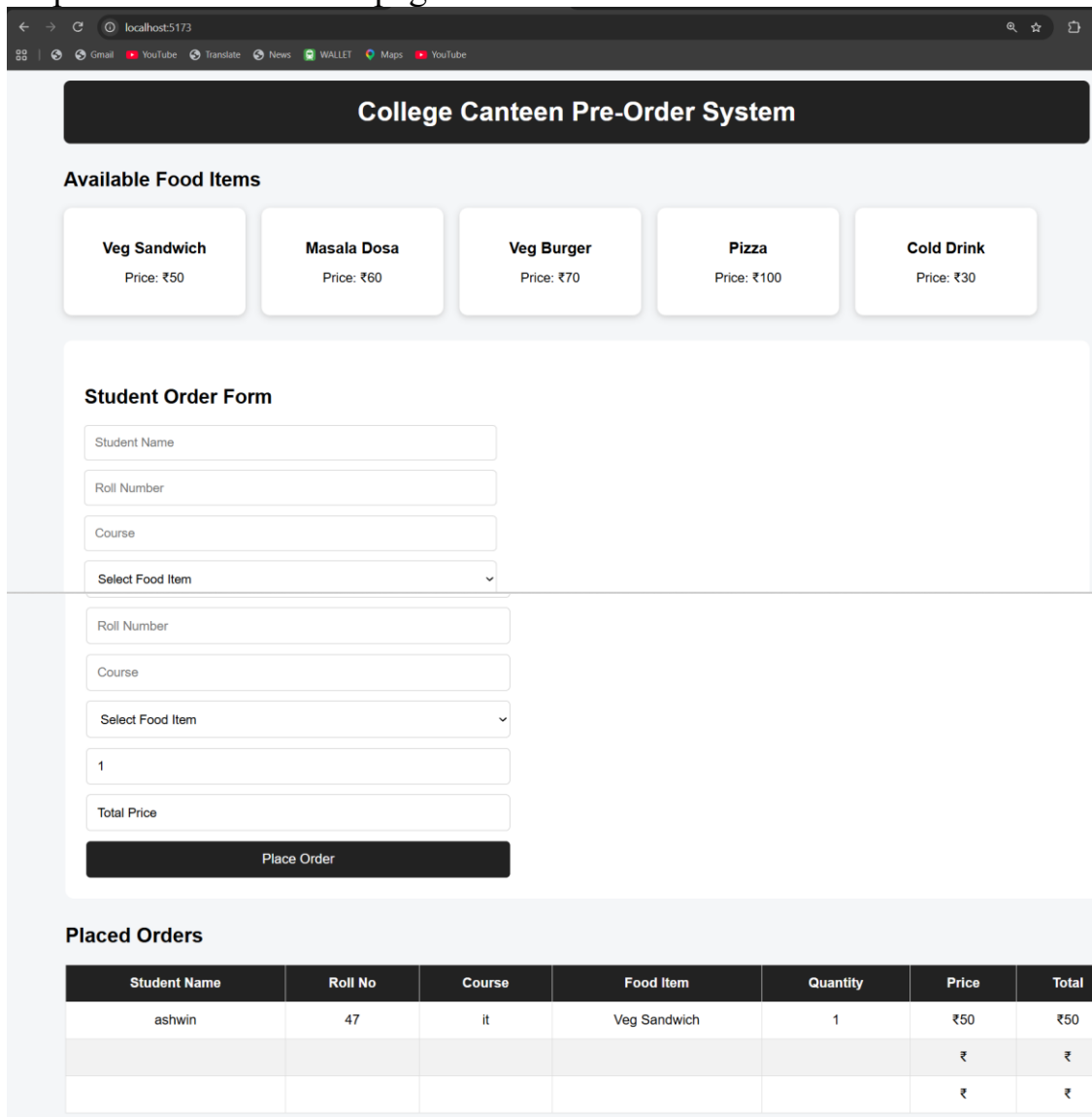
Output :



A screenshot of a web browser window displaying a JSON array of food items. The browser's address bar shows 'localhost:5000/food'. The JSON data is as follows:

```
[
  {
    "id": 1,
    "name": "Veg Sandwich",
    "price": 50
  },
  {
    "id": 2,
    "name": "Masala Dosa",
    "price": 60
  },
  {
    "id": 3,
    "name": "Veg Burger",
    "price": 70
  },
  {
    "id": 4,
    "name": "Pizza",
    "price": 100
  },
  {
    "id": 5,
    "name": "Cold Drink",
    "price": 30
  }
]
```

Step 12 : frontend server page.



A screenshot of the 'College Canteen Pre-Order System' frontend. The page features a header with the system name, a section for 'Available Food Items' with five cards (Veg Sandwich, Masala Dosa, Veg Burger, Pizza, Cold Drink) showing their prices, a 'Student Order Form' with input fields for student details and food selection, and a 'Placed Orders' table at the bottom.

College Canteen Pre-Order System

Available Food Items

- Veg Sandwich** Price: ₹50
- Masala Dosa** Price: ₹60
- Veg Burger** Price: ₹70
- Pizza** Price: ₹100
- Cold Drink** Price: ₹30

Student Order Form

Student Name

Roll Number

Course

Select Food Item

Roll Number

Course

Select Food Item

1

Total Price

Place Order

Placed Orders

Student Name	Roll No	Course	Food Item	Quantity	Price	Total
ashwin	47	it	Veg Sandwich	1	₹50	₹50
					₹	₹
					₹	₹

Step 13 : MongoDB store database..

